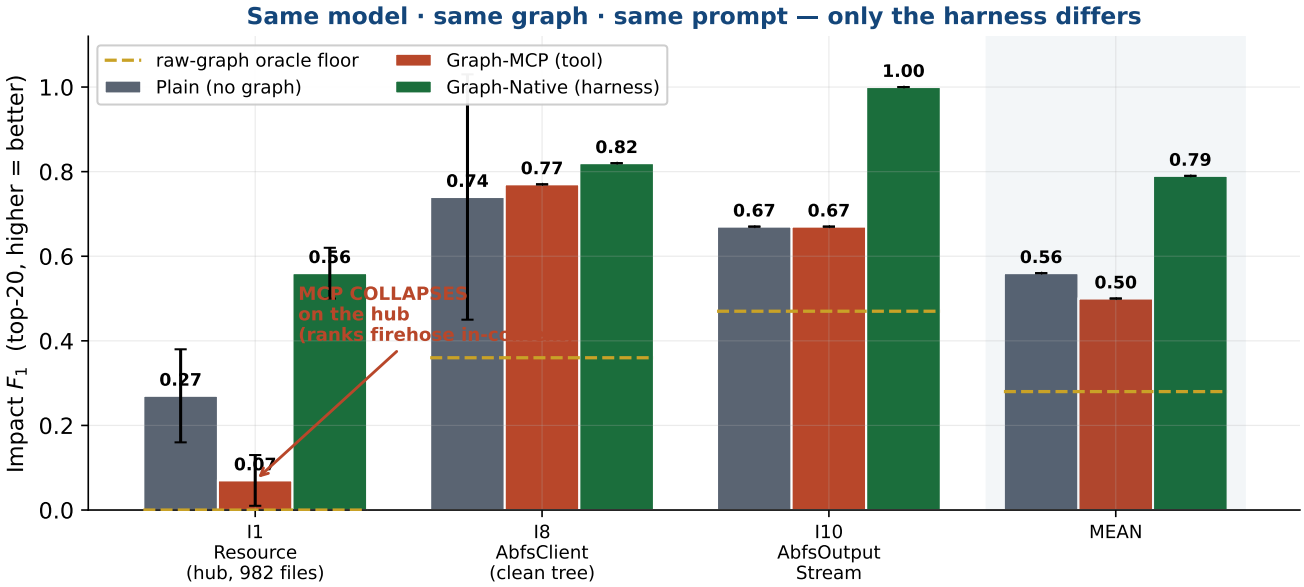
Why Graph-Native Beats Graph-as-a-Tool

A visual case study on Apache Hadoop — results, the algorithm, and how we got there

Same model (`sonnet-4.6`) · same code graph · same prompt · same Claude subscription · 3 runs/cell · only the *harness* differs

TL;DR. A coding agent can use a code graph two ways: as a *tool it calls and ranks in its own head* (Graph-MCP), or *natively*, with the harness querying and ranking the graph *before the agent's first token*. They are not equivalent. Across three real Hadoop impact-analysis tasks, graph-native scores **0.79** mean F_1 vs **0.50** for the same graph over MCP and **0.56** for no graph — *and is the cheapest arm every time*. On a hub class the MCP arm **collapses 8×** (it has the graph but can't rank a 982-file firehose mid-reasoning). **The win is not a better tool or model — it is moving the *ranking* out of the model's context and into deterministic harness code.**

1 The result



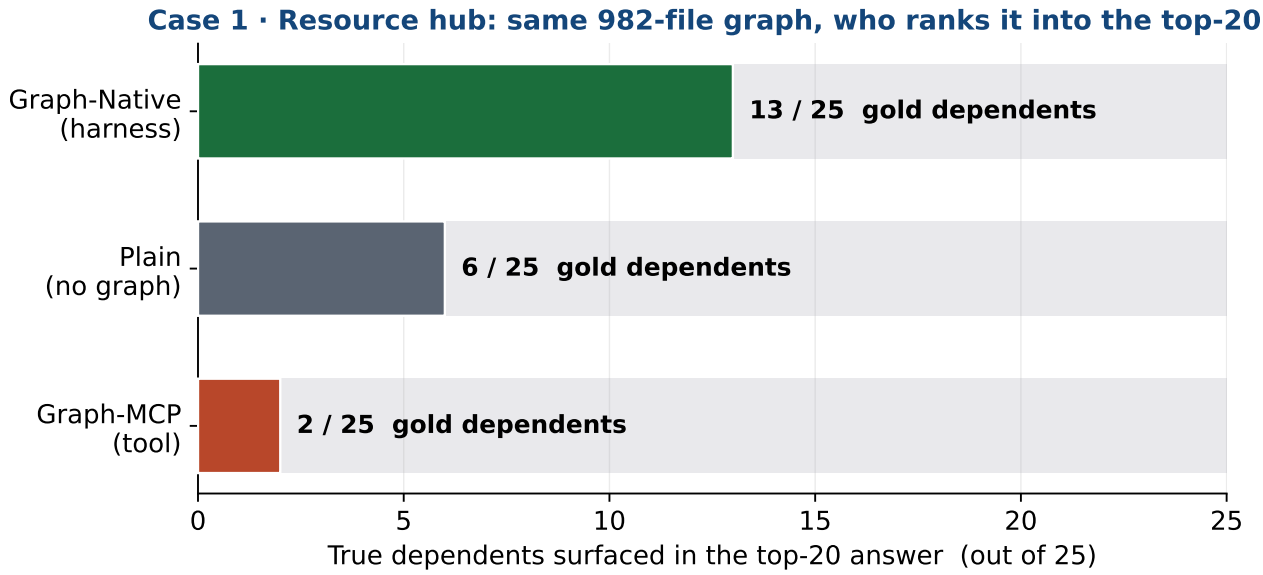
Three arms, identical except for harness wiring, scored by F_1 of the committed dependent-file list against the real PR's gold (top-20, basename match). Error bars are 95% CIs over 3 runs. Graph-native wins all three tasks, with *non-overlapping CIs* on the two decisive ones, beats the raw-graph oracle floor everywhere, and does **0 file-reads / 0 greps** on every task (Plain grep-storms 13–29 times). The margin is huge on the hub and small on the clean task — a *predictable, discriminative* pattern, not a sweep.

What each arm costs and does (mean over the 3 tasks):

	mean F_1	cost/task	Read	Grep
Plain (no graph)	0.56	$0.60	0	23
Graph-MCP (tool)	0.50	$0.39	0	0.3
Graph-Native (harness)	**0.79**	**$0.29**	0	0

2 Case 1 — the hub: why MCP collapses

The task: name the files affected by changing `Resource` (a YARN god-object). Its blast radius is **982 files** — the graph *contains* all 25 true dependents, so recall is trivial. The hard part is *ranking* the 25 that matter into the top-20. That is the whole game.



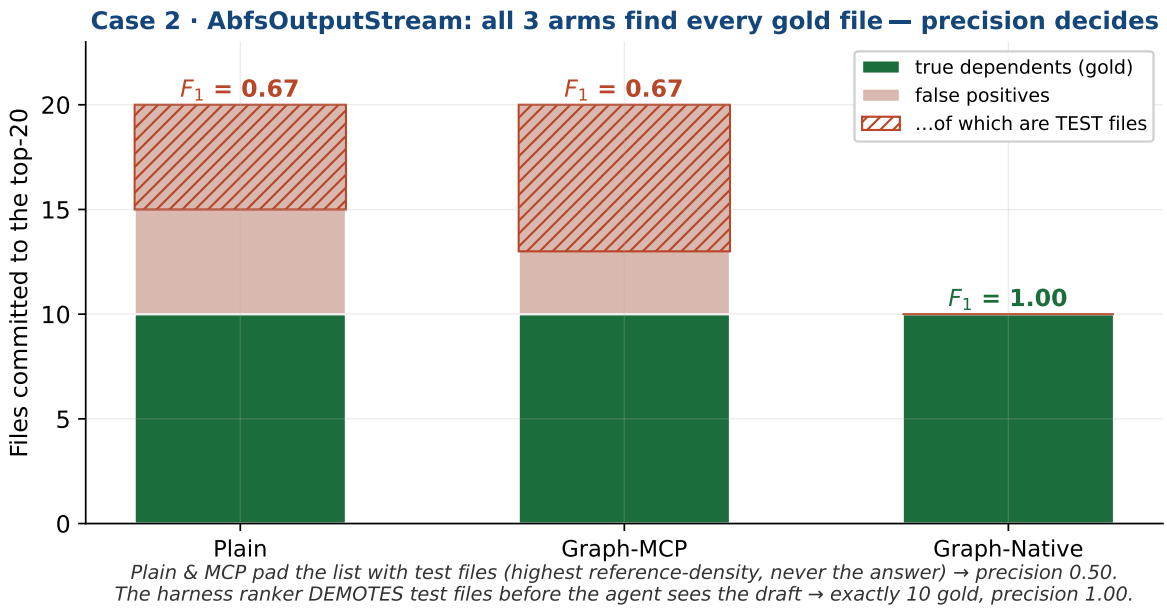
The graph CONTAINS all 25. Graph-MCP must rank the firehose mid-reasoning and finds 2; the harness pre-ranks it in code and the agent keeps 13. → F_1 0.07 vs 0.56 (8×).

All three arms see the same 982-file graph. **Graph-MCP** *calls* `impact` (7 graph calls) but, having pulled the firehose into its context, must rank it while also reasoning — and commits a short, hedged list that lands only **2 of 25** (it even names `NodeManager`, `AppInfo`, `TestResource`). **Graph-Native** starts from the harness's *pre-ranked, test-free draft* and keeps **13 of 25**. Same model, same graph, same budget; an 8× gap that is *entirely* about where the firehose got ranked.

The mechanism, in one task. A tool you have to rank in-context is not the same as an answer ranked for you in code. On a hub, "rank the firehose mid-reasoning" fails — F_1 0.07 vs 0.56.

3 Case 2 — the test-file trap: why precision needs the harness

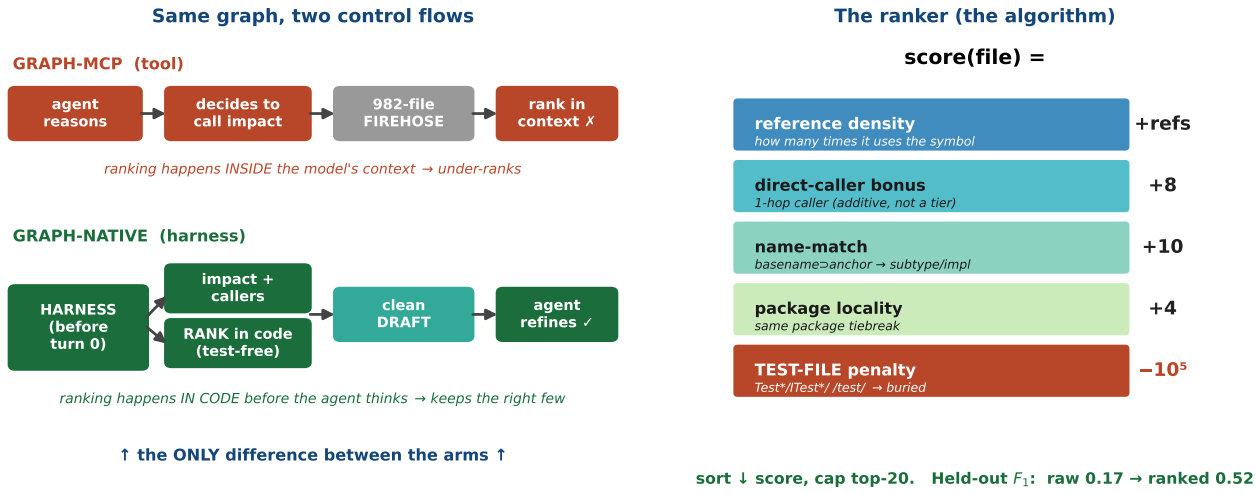
The opposite regime: `AbfsOutputStream`, a *small* blast radius (10 gold). Here *every* arm finds all 10 true dependents — recall is solved. Precision is decided by what *else* lands in the top-20.



Plain and **Graph-MCP** pad their lists with *test files* (`TestAbfsOutputStream`, `ITestAzureBlobFileSystemChecksum`, ...). Test files have the *highest* reference density on a real codebase and are *never* the answer in a production fix — so they halve precision to 0.50 (F_1 0.67). **Graph-Native** commits **exactly the 10 gold files, zero false positives** (F_1 1.00), because the harness ranker *demotes test files below every production candidate before the agent ever sees the draft.* The agent never has to relearn mid-task that a test isn't a dependent — the harness already knew.

4 The algorithm

Both effects above come from the same two-part design: *(1)* the harness runs and ranks the graph before turn 0, and *(2)* the ranker is a tiny, deterministic, gold-blind scoring function.



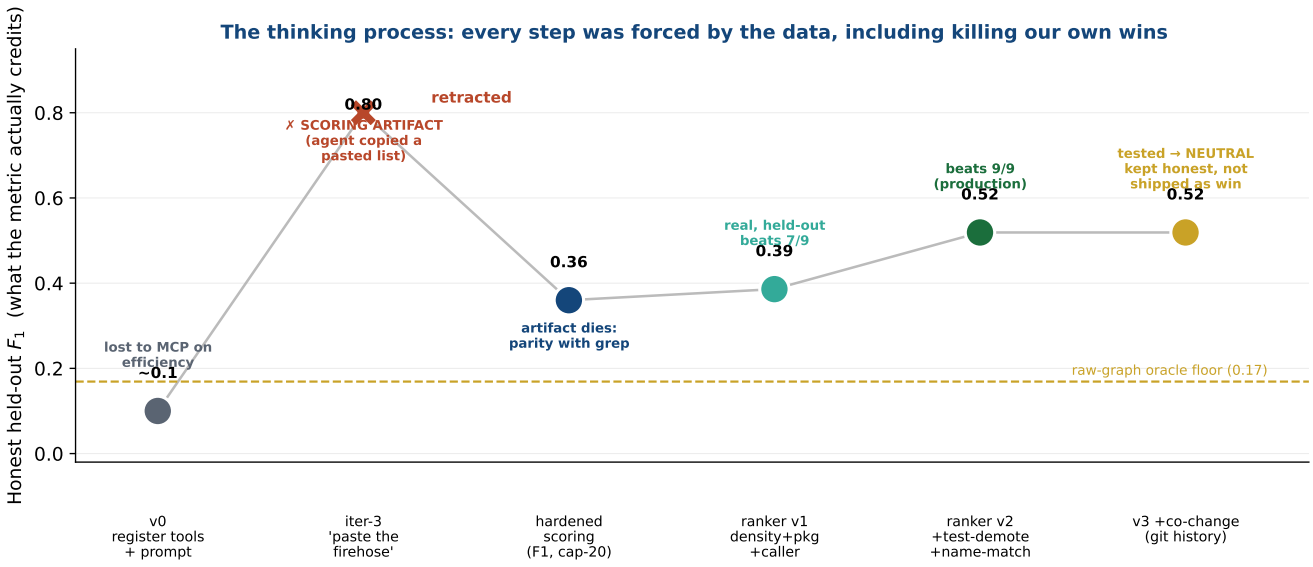
Left: the *only* difference between the arms is that ranking moves from inside the model's context (MCP) to deterministic code before the agent thinks (Native). Right: the ranker — reference density as the primary signal, an *additive* (not tiering) direct-caller bonus, a type-name-inheritance match that catches

subtype/implementor dependents with no call edge, package locality as a tiebreak, and a hard *test-file demotion*. Scored alone, with no agent in the loop, this triples the raw graph's usable quality on held-out tasks: F_1 **0.17** → **0.52**. That ranked shortlist is exactly what the harness injects at turn 0.

> **The win is a ranker, not a tool.** MCP has the identical `impact` query — it just has to rank the output itself. Relocating that ranking into 40 lines of deterministic harness code is the entire delta.

5 How we got here (the thinking process)

This did not start as a win. The honest path mattered more than any single number — including *retracting our own headline* when the data said it was fake.



- **v0** (register graph tools + a graph-first prompt) actually *lost* to MCP on efficiency. Tools + a nudge are not enough.
- **iter-3 looked like a blowout (0.80)** — until a forensic re-audit showed it was a *scoring artifact*: the harness pasted the firehose into the prompt and the agent copied it, with one tool call and zero reasoning. The old metric (substring-scan of prose) rewarded the paste. **We retracted it.**
- **Hardened scoring** (single bounded field, F1, top-20 cap, oracle floor) killed the artifact — the paste fell to 0.36, parity with grep. Now the benchmark was a *real* problem.
- **The real, unfakeable win is the ranker.** v1 (density+caller+package) reached 0.386 held-out; **v2** added test-demotion + name-match → **0.519, beating the oracle on 9/9 held-out tasks**. This is the production ranker.
- **We even rejected a good idea honestly.** A co-change signal (mine git history for files that change together with the anchor — something grep *and* static analysis lack) is a *real* signal, but at top-20 it was neutral-to-negative. We kept it as a documented opt-in and *did not* claim it as a win.

> **The discipline is the point.** Every step was forced by the data: build, validate held-out, and *delete your own win the moment a metric can fake it.* That is why the final 0.79 is trustworthy.

6 One more check: is this just our graph?

We ran the same gold against a second product, **potpie** (Neo4j + tree-sitter). Two honest findings: *(1)* potpie's *agents* need a paid API key and a multi-service Docker stack — they can't run on a subscription, while our whole graph-native arm runs from one local binary. *(2)* potpie's *raw* graph actually out-recalls ours (held-out F_1 0.38 vs our raw 0.17) — a good graph — but our *ranked* output (0.52) beats both raw graphs, and on the `Resource` hub potpie's 603-file *unranked* set scores 0.00 at top-20 while we score 0.56. **A good raw graph is necessary but not sufficient; the ranking layer — a harness job — is what turns a graph into answers.**

Reproduce: `score-matrix-ci.mjs` (the matrix), `validate-ranker-v2.mjs` (the ranker), `score-potpie-graph.mjs` (potpie), all in `bench/`. Honest caveat: these tasks measure impact *retrieval* (a ranked file list), not code synthesis; results are on one Java repo. Full write-up: `bench/ROBUSTNESS-REPORT.md`.